

week3

February 6, 2026

0.1 Week 3 – Data Preparation for Modelling (DMNN)

0.2 Task 1: Preparing the Dataset

This notebook builds on the data cleaning and exploratory analysis performed in Week 1. The goal is to prepare a clean, well-defined dataset suitable for modelling in later tasks.

```
[1]: import pandas as pd
import numpy as np

df = pd.read_csv("online_retail_II.csv", encoding="latin1")
df.head()
```

```
[1]: Invoice StockCode Description Quantity \
0 489434 85048 15CM CHRISTMAS GLASS BALL 20 LIGHTS 12
1 489434 79323P PINK CHERRY LIGHTS 12
2 489434 79323W WHITE CHERRY LIGHTS 12
3 489434 22041 RECORD FRAME 7" SINGLE SIZE 48
4 489434 21232 STRAWBERRY CERAMIC TRINKET BOX 24

InvoiceDate Price Customer ID Country
0 01/12/2009 07:45 6.95 13085.0 United Kingdom
1 01/12/2009 07:45 6.75 13085.0 United Kingdom
2 01/12/2009 07:45 6.75 13085.0 United Kingdom
3 01/12/2009 07:45 2.10 13085.0 United Kingdom
4 01/12/2009 07:45 1.25 13085.0 United Kingdom
```

Data Cleaning

Based on the Week 1 analysis, the following decisions are applied:

- Rows with missing or invalid Customer IDs are removed
- Cancelled transactions (negative quantities) are excluded
- Extreme values are retained, as they reflect genuine high-volume purchases
- InvoiceDate is parsed as a datetime variable

```
[2]: df = df.dropna(subset=["Customer ID"])
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")
df = df[df["Quantity"] > 0]
df = df[df["Price"] > 0]
```

```
df.shape
```

```
[2]: (407664, 8)
```

Unit of Analysis

The invoice (basket) level is used as the unit of analysis. This avoids treating individual invoice lines as independent observations.

```
[3]: df["Revenue"] = df["Quantity"] * df["Price"]

invoice_df = (
    df.groupby("Invoice")
      .agg(
        invoice_date=("InvoiceDate", "min"),
        customer_id=("Customer ID", "first"),
        basket_size=("Quantity", "sum"),
        invoice_value=("Revenue", "sum")
      )
      .reset_index()
)

invoice_df.head()
```

```
[3]:   Invoice      invoice_date  customer_id  basket_size  invoice_value
0  489434  2009-01-12 07:45:00      13085.0           166           505.30
1  489435  2009-01-12 07:46:00      13085.0            60           145.80
2  489436  2009-01-12 09:06:00      13078.0           193           630.33
3  489437  2009-01-12 09:08:00      15362.0           145           310.75
4  489438  2009-01-12 09:24:00      18102.0           826          2286.24
```

Outcome of Task 1:

The dataset has been cleaned, aggregated, and structured at the invoice level. It is now suitable for feature engineering and modelling in subsequent tasks.

0.3 Task 2 - Define a Target Variable (Critical Thinking)

```
[4]: revenue_threshold = invoice_df["invoice_value"].quantile(0.75)

invoice_df["high_value_invoice"] = (
    invoice_df["invoice_value"] >= revenue_threshold
).astype(int)

invoice_df["high_value_invoice"].value_counts(normalize=True)
```

```
[4]: high_value_invoice
0    0.749961
1    0.250039
```

Name: proportion, dtype: float64

```
[5]: invoice_df[["invoice_value", "high_value_invoice"]].head()
```

```
[5]:   invoice_value  high_value_invoice
0         505.30                1
1         145.80                0
2         630.33                1
3         310.75                0
4        2286.24                1
```

Target Variable Definition

The modelling problem is formulated as a **binary classification task**. The objective is to predict whether an invoice represents a high-value transaction.

The target variable, `high_value_invoice`, is constructed using the empirical distribution of invoice revenue. Invoices with total revenue in the top 25% of all invoices are labelled as high value (1), while the remaining invoices are labelled as regular (0).

This quantile-based threshold avoids optional cut-off values and reflects relative transaction value within the dataset.

Risk / ambiguity: The definition of a high-value invoice depends on the chosen quantile threshold. Alternative thresholds would change class balance and may influence model performance.

0.4 Task 3 – Feature Readiness for Modelling

This task evaluates whether the invoice-level dataset created in Task 1 and labelled in Task 2 is suitable for use in supervised machine learning models.

```
[6]: invoice_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19213 entries, 0 to 19212
Data columns (total 6 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Invoice              19213 non-null  object
1   invoice_date        8010 non-null   datetime64[ns]
2   customer_id         19213 non-null  float64
3   basket_size         19213 non-null  int64
4   invoice_value       19213 non-null  float64
5   high_value_invoice  19213 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(2), object(1)
memory usage: 900.7+ KB
```

```
[7]: invoice_df["log_invoice_value"] = np.log(invoice_df["invoice_value"]).
      ↪clip(lower=1)
invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
```

```
invoice_df[["invoice_value", "log_invoice_value", "weekday"]].head()
```

```
[7]:   invoice_value  log_invoice_value  weekday
0         505.30         6.225152        0.0
1         145.80         4.982236        0.0
2         630.33         6.446243        0.0
3         310.75         5.738989        0.0
4        2286.24         7.734664        0.0
```

Feature Overview

The dataset contains the following feature groups:

- Identifiers:
 - `Invoice` (invoice identifier)
 - `customer_id` (customer identifier)
- Temporal:
 - `invoice_date` (datetime)
- Categorical:
 - `country`
- Numerical:
 - `total_quantity`
 - `total_revenue`
 - `num_products`

The target variable `high_value_invoice` is binary and was defined in Task 2.

Assumptions and Risks

Assumptions: - Each invoice indicates an independent decision regarding purchasing. - Aggregated invoice features adequately summarise customer behaviour. - Extreme invoice values provide indicate genuine large purchases rather than data errors.

Risks: - Customer purchasing behaviour may vary over time, violating independence assumptions. - Country information is highly imbalanced, which may bias models. - Revenue distributions are strongly right-skewed and may require transformation.

Modelling Readiness Assessment

- Numerical features are directly usable by most machine learning models.
- Categorical features (e.g. `country`) will require encoding (e.g. one-hot encoding).
- Datetime features (`invoice_date`) cannot be used directly and may require feature extraction (e.g. day, month, recency).
- Identifier variables (`Invoice`, `customer_id`) should not be used as predictors but may be useful for grouping or splitting strategies.

0.4.1 Conclusion

The dataset is structurally suitable for modelling, with clearly defined numerical, categorical, and temporal features. However, appropriate encoding, feature transformation, and careful validation strategies will be required before training predictive models.

0.4.2 Task 4 — Train Tree-Based Models

```
[8]: X = invoice_df[[
        "basket_size",
        "invoice_value",
        "log_invoice_value",
        "weekday"
    ]]
y = invoice_df["high_value_invoice"]

invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
invoice_df["weekday"] = invoice_df["weekday"].fillna(-1).astype(int)

from sklearn.model_selection import train_test_split
feature_cols = ["basket_size", "invoice_value", "log_invoice_value", "weekday"]
X = invoice_df[feature_cols].copy()
y = invoice_df["high_value_invoice"].astype(int)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
from sklearn.ensemble import GradientBoostingClassifier
gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb_model.fit(X_train, y_train)
```

```
[8]: GradientBoostingClassifier(random_state=42)
```

```
[9]: from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```
[9]: RandomForestClassifier(max_depth=5, random_state=42)
```

```
[10]: from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(
    n_estimators=100,
    random_state=42
)
```

```
gb_model.fit(X_train, y_train)
```

```
[10]: GradientBoostingClassifier(random_state=42)
```

```
[11]: from sklearn.tree import DecisionTreeClassifier
      from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

      dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
      rf_model = RandomForestClassifier(n_estimators=100, max_depth=5,
      ↪random_state=42)
      gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)

      dt_model.fit(X_train, y_train)
      rf_model.fit(X_train, y_train)
      gb_model.fit(X_train, y_train)
```

```
[11]: GradientBoostingClassifier(random_state=42)
```

```
[12]: print("DT n_estimators exists?", hasattr(dt_model, "n_estimators"))
      print("RF n_estimators:", rf_model.n_estimators)
      print("GB n_estimators:", gb_model.n_estimators)
```

```
DT n_estimators exists? False
```

```
RF n_estimators: 100
```

```
GB n_estimators: 100
```

```
[13]: from sklearn.metrics import classification_report

      print("Decision Tree")
      print(classification_report(y_test, dt_model.predict(X_test)))

      print("\nRandom Forest")
      print(classification_report(y_test, rf_model.predict(X_test)))

      print("\nGradient Boosting")
      print(classification_report(y_test, gb_model.predict(X_test)))
```

```
Decision Tree
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3603
1	1.00	1.00	1.00	1201
accuracy			1.00	4804
macro avg	1.00	1.00	1.00	4804
weighted avg	1.00	1.00	1.00	4804

Random Forest

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3603
1	1.00	1.00	1.00	1201
accuracy			1.00	4804
macro avg	1.00	1.00	1.00	4804
weighted avg	1.00	1.00	1.00	4804

Gradient Boosting

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3603
1	1.00	1.00	1.00	1201
accuracy			1.00	4804
macro avg	1.00	1.00	1.00	4804
weighted avg	1.00	1.00	1.00	4804

Model comparison

All three tree-based models were trained using identical features and data splits to allow fair comparison.

The single decision tree provides a transparent baseline but is limited in expressive power. The random forest improves stability by aggregating multiple trees. The gradient boosting model captures more complex patterns through sequential learning.

At this stage, the objective is structural comparison rather than performance optimisation.

Preprocessing considerations

No feature scaling was applied, as tree-based models are invariant to monotonic transformations. Categorical variables were excluded to avoid introducing encoding complexity at this stage.

Note on Performance Interpretation

The perfect classification performance observed across all models is driven by target leakage, as the target variable is directly derived from `total_revenue`, which is also included as a predictor.

This task is therefore used to demonstrate model structure and behaviour rather than to evaluate real-world predictive performance.

Task 5 — Validation-Based Comparison

```
[14]: from sklearn.model_selection import train_test_split

X_temp, X_test, y_temp, y_test = train_test_split(
```

```

    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp
)
dt_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
gb_model.fit(X_train, y_train)

```

```
[14]: GradientBoostingClassifier(random_state=42)
```

```
[15]: from sklearn.metrics import accuracy_score

def evaluate(model, X_tr, y_tr, X_va, y_va, X_te, y_te):
    return {
        "train": accuracy_score(y_tr, model.predict(X_tr)),
        "validation": accuracy_score(y_va, model.predict(X_va)),
        "test": accuracy_score(y_te, model.predict(X_te))
    }

dt_scores = evaluate(dt_model, X_train, y_train, X_val, y_val, X_test, y_test)
rf_scores = evaluate(rf_model, X_train, y_train, X_val, y_val, X_test, y_test)
gb_scores = evaluate(gb_model, X_train, y_train, X_val, y_val, X_test, y_test)

dt_scores, rf_scores, gb_scores

```

```
[15]: ({'train': 1.0, 'validation': 1.0, 'test': 1.0},
      {'train': 1.0, 'validation': 1.0, 'test': 1.0},
      {'train': 1.0, 'validation': 1.0, 'test': 1.0})

```

0.5 Validation-Based Comparison

All three models achieve identical performance across training, validation, and test sets. This indicates no observable generalisation gap between training and validation data.

0.5.1 Model Behaviour and Complexity

- **Decision Tree**

The single tree memorises simple decision rules and provides a transparent baseline. Its limited depth restricts expressive power.

- **Random Forest**

The ensemble reduces variance by aggregating multiple trees, increasing stability. Despite higher complexity, no performance gain is observed due to target leakage.

- **Gradient Boosting**

The boosted model sequentially refines decision boundaries and is the most expressive. However, additional complexity does not translate into improved validation performance.

0.5.2 Interpretation of Train–Validation Gap

The absence of a performance gap is driven by target leakage, as the target variable is directly derived from features used for training.

Therefore, validation is used here to compare structural behaviour rather than predictive capability.

0.5.3 Task 6 — Final Test-Set Check

Why the test set is used only once

The test set is reserved as an unbiased estimate of final model performance and must not influence modelling decisions. Using the test set repeatedly risks information leakage, where model choices are indirectly optimised to the test data, leading to over-optimistic performance estimates. Therefore, all modelling decisions were finalised using training and validation data before evaluating each model once on the test set.

Alignment with validation behaviour

Test-set performance aligns fully with validation results for all three tree-based models. No performance degradation or unexpected behaviour is observed, indicating that model behaviour is stable across data splits. Small numerical differences are not interpreted, as the objective is structural comparison rather than performance optimisation.

Interpretation

The consistent performance across training, validation, and test sets reflects the simplicity of the modelling setup and the strong relationship between engineered features and the target variable. The test results confirm earlier validation observations without contradicting them.

[]: